

Instructions: 42 marks | 70 min | **Set A** | MCQs: 10×3=30 marks (one correct, −1 wrong) | Subjective: 2×6=12 marks

Quick Reference

requests:

`.status_code` (int)
`.ok` (True if 2xx)
`.json()` / `.text`

HTTP Codes:

200 OK, 201 Created
 401 Unauth, 404 NotFound
 429 TooManyRequests

Pydantic:

`ge/le`: $\geq$ / $\leq$, `gt/lt`: $>$ / $<$
`min_length`: min str len

jq:

`.field` access
`.arr[]` iterate
`select()`, `map()`
`| length` count

Formulas:

$$P_o = \frac{\text{agreed}}{\text{total}}$$

$$u = 1 - \max(P_i)$$

$$P_{agree} = \alpha_1 \alpha_2 + (1 - \alpha_1)(1 - \alpha_2)$$

1. (3 points) [MCQ] Calculate **Cohen's Kappa** (κ) for this confusion matrix:

	B: Pos	B: Neg	Total
A: Pos	80	20	100
A: Neg	30	70	100
Total	110	90	200

- A. **0.50**
- B. 0.60
- C. 0.45
- D. 0.55
- E. 0.40
- F. 0.35
- G. 0.80
- H. 0.25
- I. 0.75
- J. 0.70

Solution

$P_o = 150/200 = 0.75$. $P_e = (100/200)(110/200) + (100/200)(90/200) = 0.275 + 0.225 = 0.50$.
 $\kappa = (0.75 - 0.50)/(1 - 0.50) = 0.25/0.50 = 0.50$. Answer: **0.50**

2. (3 points) [MCQ] In **batch-mode** active learning, you select 3 samples to label. Using only uncertainty sampling, you pick the 3 most uncertain. What is the main problem with this approach?
- A. The model will overfit to uncertain regions
 - B. The batch size is too small
 - C. The samples may be too easy for the model
 - D. **The selected samples may be too similar (lack diversity)**
 - E. Annotators cannot label multiple samples at once
 - F. Batch selection requires more memory than sequential
 - G. Uncertainty is expensive to compute
 - H. Uncertain samples are always outliers
 - I. The model needs to be retrained after each sample
 - J. Uncertainty cannot be computed for unlabeled data

Solution

Pure uncertainty sampling in batch mode often selects samples from the same region of uncertainty, providing redundant information. Diversity sampling or hybrid approaches (uncertainty + diversity) address this. Answer: **Lack diversity / too similar**

3. (3 points) [MCQ] What is the output of this code?

```
1 from bs4 import BeautifulSoup
2 html = """
3 <div class="movie"><span class="rating">8.8</span></div>
4 <div class="movie"><span class="year">2009</span></div>
5 """
6 soup = BeautifulSoup(html, 'html.parser')
7 movies = soup.select('.movie')
8 print(len(movies[1].select('.rating')))
```

- A. "0"
- B. 2009
- C. 8.8
- D. 0
- E. 2
- F. .rating
- G. 1
- H. None
- I. Error
- J. []

Solution

`movies[1]` is the second div (has `.year`, not `.rating`). `select('.rating')` returns empty list. `len([]) = 0`. Answer: **0**

4. (3 points) [MCQ] What does this code print?

```
1 import requests
2 response = requests.get("https://api.example.com/data")
3 # API returns: {"items": [{"id": 1}, {"id": 2}], "total": 2}
4 # Status code: 201
5
6 if response.status_code == 200:
7     print("OK")
8 elif response.ok:
9     print("Created")
10 else:
11     print("Error")
12 print(response.json()["items"][0]["id"])
```

- A. JSONDecodeError
- B. Created, then KeyError
- C. Created, then IndexError
- D. Created, then 2
- E. OK, then 1
- F. Error, then 1
- G. Created, then {"id": 1}
- H. Error, then None
- I. OK, then ["id": 1, "id": 2]
- J. Created, then 1

Solution

Status 201 $\neq$ 200, so first `if` fails. But `response.ok` is `True` for any 2xx status (200-299), so prints "Created". Then `["items"][0]["id"]` navigates: items array $\rightarrow$ first element $\rightarrow$ id field = 1. Answer: **Created, then 1**

5. (3 points) [MCQ] How many movies match this jq filter?

```
1 cat data.json | jq '.movies[] | select(.year > 2010)'
```

data.json:

```
1 {"movies": [{"title": "A", "year": 2015}, {"title": "B", "year": 2008},
2               {"title": "C", "year": 2020}, {"title": "D", "year": 2005}],
3   "count": 4}
```

- A. Error: cannot iterate over object
- B. 0
- C. 3
- D. 2**
- E. Error: invalid filter
- F. 5
- G. All 4 (select has no effect)
- H. 4
- I. 1
- J. 8

Solution

.movies[] iterates the 4 movies. select(.year > 2010) keeps only those with year > 2010: A (2015) and C (2020). Movies B (2008) and D (2005) are filtered out. Answer: **2**

6. (3 points) [MCQ] Calculate the **IoU** (Intersection over Union) between these bounding boxes:

Box	x1	y1	x2	y2
Predicted	0	0	30	20
Ground Truth	20	0	50	20

- A. 0.40
- B. 0.33
- C. 0.30
- D. 0.17
- E. 0.67
- F. 0.20**
- G. 0.50
- H. 0.25
- I. 0.10
- J. 0.15

Solution

Pred: $30 \times 20 = 600$. GT: $30 \times 20 = 600$. Intersection: $10 \times 20 = 200$.
 Union = $600 + 600 - 200 = 1000$. IoU = $200/1000 = 0.20$. Answer: **0.20**

7. (3 points) [MCQ] You have a labeled validation set to evaluate two labeling functions:

Sample	True Label	LF1	LF2
1	HIT	HIT	ABSTAIN
2	HIT	HIT	HIT
3	FLOP	HIT	FLOP
4	FLOP	ABSTAIN	FLOP
5	HIT	FLOP	HIT

On samples where **both LFs vote AND disagree**, what fraction of the time is **LF2 correct**?

- A. 0.00
- B. 0.50

- C. 0.80
- D. 1.00**
- E. 0.40
- F. 0.33
- G. 0.25
- H. 0.75
- I. Cannot determine
- J. 0.67

Solution

First find samples where both vote (no ABSTAIN): 2, 3, 5.
 Then find where they disagree: Sample 3 (HIT≠FLOP), Sample 5 (FLOP≠HIT).
 Check LF2 correctness on disagreements:
 Sample 3: LF2=FLOP, True=FLOP ✓ Sample 5: LF2=HIT, True=HIT ✓
 LF2 correct on 2/2 disagreements = **1.00**. (LF2 is more reliable when they conflict!)

8. (3 points) [MCQ] Which input **passes** this Pydantic validation?

```
1 class Movie(BaseModel):
2     title: str = Field(min_length=1)
3     year: int = Field(ge=1888, le=2025)
4     runtime: int = Field(gt=0)
```

- A. Movie(title="", year=1900, runtime=100)
- B. Movie(title="", year=2020, runtime=120)
- C. Movie(title="Dune", year=1887, runtime=90)
- D. Movie(title="Dune", year=1888, runtime=0)
- E. Movie(title="Dune", year=2025, runtime=-1)
- F. Movie(title="A", year=2026, runtime=120)
- G. Movie(title="Dune", year=2025, runtime=0)
- H. Movie(title="Dune", year=2021, runtime=0)
- I. Movie(title="Dune", year=1888, runtime=1)**
- J. Movie(title="Dune", year=2030, runtime=155)

Solution

Constraints: title min_length=1, year in [1888, 2025], runtime > 0.
Fails: title="" (empty), year=2030/2026/1887 (out of range), runtime=0/-1 (not > 0).
Passes: title="Dune", year=1888 (ge means ≥), runtime=1 (gt means >).

9. (3 points) [MCQ] API rate limit: **100 req/min**. Each response returns **5 movies**. You need **500 movies**. Minimum time?

- A. 10 min
- B. 0.1 min
- C. 5 min
- D. 2 min
- E. 30 sec
- F. 6 min
- G. 50 sec
- H. 3 min
- I. 1 min**
- J. 100 sec

Solution

Requests = $\lceil \text{movies} / \text{per_resp} \rceil$. Time = $\lceil \text{requests} / \text{rate} \rceil$.
 Set A: $\lceil 500 / 5 \rceil = 100$ requests, $100 / 100 = 1$ min. Answer: **1 min**

10. (3 points) [MCQ] You have this JSON and want to convert to CSV. What is the main challenge?

```
1 {"movies": [
2   {"title": "Dune", "genres": ["Sci-Fi", "Action"], "rating": 8.0},
3   {"title": "Joker", "genres": ["Drama"], "rating": 8.4}
4 ]}
```

- A. JSON keys become CSV headers automatically
- B. The movies key name is too generic
- C. JSON arrays must be sorted before conversion
- D. The rating field has inconsistent types across rows
- E. The genres array has variable length across rows**
- F. JSON whitespace affects the conversion
- G. The title field contains special characters
- H. CSV cannot store floating-point numbers

Solution

CSV is flat (rows/columns). The **genres** array has variable length (2 vs 1) and cannot directly become fixed columns. Options: flatten to string ("Sci-Fi|Action"), create separate rows, or use max columns. Answer: **Variable-length arrays**

11. (6 points) [Subjective] You have 5 unlabeled samples on a 1D line. One sample (L) at position 0 is already labeled. Use acquisition function: $a(x) = u(x) + \lambda \cdot d(x)$ where $\lambda = 1$ and $d(x)$ is distance to nearest labeled sample.

Sample	Position	Uncertainty $u(x)$
A	1	0.9
B	2	0.8
C	4	0.6
D	5	0.5
E	3	0.7

- (a) (2 marks) Calculate $d(x, L)$ and $a(x)$ for all samples. Which sample is selected first?
- (b) (2 marks) After selecting that sample, recalculate $d'(x) = \min(d(x, L), d(x, \text{selected}))$ for remaining samples. Which is selected second?
- (c) (2 marks) Compare $\lambda = 0$ vs $\lambda = 10$: which sample would each select? Which λ prioritizes *covering unexplored regions* vs *labeling uncertain samples*?

Solution

(a) Distance $d(x, L) = |\text{pos} - 0| = \text{pos}$. Then $a(x) = u(x) + 1 \cdot d(x, L)$:
 A: $0.9 + 1 = 1.9$ B: $0.8 + 2 = 2.8$ C: $0.6 + 4 = 4.6$ D: $0.5 + 5 = 5.5$ E: $0.7 + 3 = 3.7$
 Select **D** (highest score 5.5).
 (b) D is at position 5. New $d'(x) = \min(|\text{pos} - 0|, |\text{pos} - 5|)$:
 A: $\min(1, 4) = 1$ B: $\min(2, 3) = 2$ C: $\min(4, 1) = 1$ E: $\min(3, 2) = 2$
 New scores: A: $0.9 + 1 = 1.9$, B: $0.8 + 2 = 2.8$, C: $0.6 + 1 = 1.6$, E: $0.7 + 2 = 2.7$
 Select **B** (score 2.8).
 (c) $\lambda = 0$: $a(x) = u(x)$ only $\rightarrow$ selects **A** (highest uncertainty 0.9). Prioritizes uncertain samples.
 $\lambda = 10$: $a(x) = u(x) + 10d(x) \rightarrow$ selects **D** (score $0.5 + 50 = 50.5$). Prioritizes unexplored regions (far from L).
 Higher λ = more weight on distance = covering unexplored regions.

12. (6 points) [Subjective] Given these weak supervision labeling functions:

```

1 # All amounts in crores (INR)
2 def lf_high_rating(movie):
3     return HIT if movie.rating >= 8.0 else ABSTAIN
4
5 def lf_low_budget_profit(movie):
6     if movie.budget < 10 and movie.box_office > 50: # crores
7         return HIT
8     return ABSTAIN
9
10 def lf_poor_reviews(movie):
11     return FLOP if movie.rotten_tomatoes < 30 else ABSTAIN

```

- (a) (2 marks) For a movie with: rating=8.5, budget=5cr, box_office=100cr, rotten_tomatoes=85
What does each LF output? What is the likely final label?
- (b) (2 marks) Explain the “cold start” problem in active learning. How does weak supervision help?
- (c) (2 marks) If LF1 and LF2 agree on 40 out of 50 samples where both vote, write the equation relating their accuracies α_1 and α_2 . Why do we need ≥ 3 LFs to solve for individual accuracies?

Solution

(a)

- lf_high_rating: $8.5 \geq 8.0 \rightarrow \text{HIT}$
- lf_low_budget_profit: $5\text{M} < 10\text{M}$ and $100\text{M} > 50\text{M} \rightarrow \text{HIT}$
- lf_poor_reviews: $85 \geq 30 \rightarrow \text{ABSTAIN}$

Final: 2 HIT, 0 FLOP, 1 ABSTAIN $\rightarrow \text{HIT}$

(b) Cold start: Initial model trained on few samples has poor predictions $\rightarrow$ uncertainty estimates unreliable $\rightarrow$ AL selects wrong samples. Weak supervision provides initial (noisy) labels without human effort, giving AL a better starting model.

(c) $P(\text{agree}) = \alpha_1\alpha_2 + (1 - \alpha_1)(1 - \alpha_2) = \frac{40}{50} = 0.8$

This is 1 equation with 2 unknowns $\rightarrow$ infinite solutions. With 3 LFs, we get 3 pairwise agreement equations with 3 unknown accuracies $\rightarrow$ solvable system.